

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO5 – Develop a Program Based on Control Flow Statements

Nominal Duration: Theory 3.5 Hours / Practical 12 Hours

L.O 05: DEVELOP A PROGRAM BASED ON CONTROL FLOW STATEMENTS

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Explain Python decision-making and control flow.
2. Identify logical conditions used in Python programs.
3. Identify and apply comparison operators.
4. Identify and apply logical operators.
5. Develop programs using `if` statements.
6. Develop programs using `if-else` statements.
7. Develop programs using `if-elif-else` statements.
8. Apply shorthand `if` statements.
9. Apply shorthand `if-else` conditional expressions.
10. Initialize and assign values to variables.
11. Compare two variables using conditional statements.
12. Display output when specified conditions are true or false.
13. Apply multiple conditions in decision-making.
14. Apply nested conditional statements.
15. Apply the `pass` statement where appropriate.
16. Draw process flow diagrams for conditional statements.
17. Test and debug control-flow programs.
18. Apply conditional statements to simple practical problems.

INFORMATION SHEET No. 5.1

Title: Understanding Python Control Flow and Conditional Statements

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain decision-making and control flow.
- Identify logical conditions.
- Identify comparison operators.
- Identify logical operators.
- Explain the syntax of `if`.
- Explain `if-else`.
- Explain `if-elif-else`.
- Apply multiple conditions.
- Explain shorthand conditional statements.
- Explain nested conditions.
- Explain the `pass` statement.
- Initialize and assign variables for conditional evaluation.
- Explain basic flowchart symbols.
- Draw decision-making process flow diagrams.
- Apply conditional statements to practical problems.

1. Introduction to Control Flow

Control flow refers to the order in which instructions in a computer program are executed.

Normally, Python executes statements sequentially from top to bottom.

Example:

```
print("Step 1")
print("Step 2")
print("Step 3")
```

Output:

```
Step 1
Step 2
Step 3
```

Sometimes a program must make a decision.

For example:

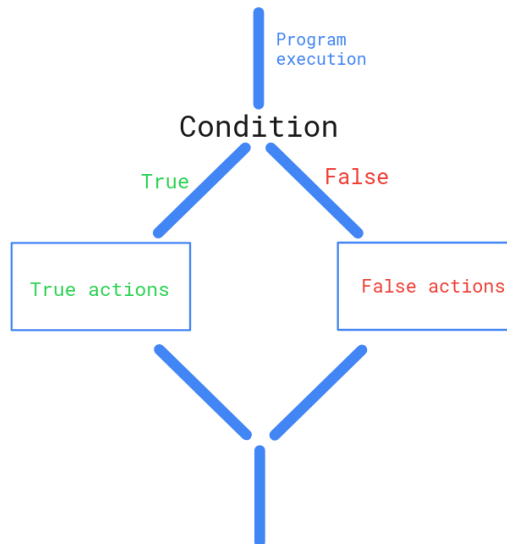
- Is the student score greater than or equal to 50?
- Is a person 18 years old or older?
- Is the password correct?
- Is `b` greater than `a`?
- Is a number positive, negative, or zero?

Conditional statements allow Python to make these decisions.

2. Decision-Making in Python

Decision-making means evaluating a condition and performing an action according to whether that condition is True or False.

General concept:



Example:

```
age = 20

if age >= 18:
    print("Adult")
```

Python evaluates:

```
age >= 18
```

If the result is True, Python executes:

```
print("Adult")
```

3. Logical Conditions

A logical condition is an expression that produces:

True

or:

False

Example:

```
a = 5
b = 10

print(b > a)
```

Output:

True

The expression:

`b > a`

is a logical condition.

4. Comparison Operators

Comparison operators compare two values.

Operator	Description	Example
<code>==</code>	Equal to	<code>a == b</code>
<code>!=</code>	Not equal to	<code>a != b</code>
<code>></code>	Greater than	<code>a > b</code>
<code><</code>	Less than	<code>a < b</code>
<code>>=</code>	Greater than or equal to	<code>a >= b</code>
<code><=</code>	Less than or equal to	<code>a <= b</code>

Example:

```
a = 5
b = 10

print(a == b)
print(a != b)
print(a < b)
print(a > b)
print(a <= b)
print(a >= b)
```

Output:

```
False
True
True
False
True
False
```

5. Logical Operators

Logical operators combine or modify conditions.

Python provides three main logical operators:

Operator	Description
<code>and</code>	True when both conditions are true
<code>or</code>	True when at least one condition is true
<code>not</code>	Reverses the condition

6. The and Operator

The and operator requires both conditions to be true.

Example:

```
age = 20
score = 80

if age >= 18 and score >= 50:
    print("Eligible")
```

Both conditions must be true.

Truth Table

Condition A	Condition B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

7. The or Operator

The or operator requires at least one condition to be true.

Example:

```
score = 45
attendance = 90

if score >= 50 or attendance >= 80:
    print("Condition satisfied")
```

Truth Table

Condition A	Condition B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

8. The not Operator

The not operator reverses a Boolean result.

Example:

```
is_blocked = False

if not is_blocked:
    print("Access allowed")
```

If:

```
is_blocked = False
```

then:

```
not is_blocked
```

becomes:

```
True
```

9. The if Statement

The if statement executes a block of code when a condition is true.

Syntax:

```
if condition:  
    statement
```

Example:

```
a = 5  
b = 10  
  
if b > a:  
    print("b is greater than a")
```

Output:

```
b is greater than a
```

This example directly addresses the LO5 assessment requirement.

10. Structure of an if Statement

```
if condition:  
    statement
```

Important components are:

1. if keyword
2. Condition
3. Colon :
4. Indented code block

Example:

```
score = 80  
  
if score >= 50:  
    print("Pass")
```

11. Indentation in Conditional Statements

Python uses indentation to define the statements belonging to a condition.

Correct:

```
age = 20
```

```
if age >= 18:  
    print("Adult")
```

Incorrect:

```
age = 20  
  
if age >= 18:  
print("Adult")
```

The second example causes an indentation error.

A standard Python indentation level normally uses four spaces.

12. The if-else Statement

An if-else structure allows the program to choose between two alternatives.

Syntax:

```
if condition:  
    statement_if_true  
else:  
    statement_if_false
```

Example:

```
age = 16  
  
if age >= 18:  
    print("Adult")  
else:  
    print("Minor")
```

Output:

Minor

13. if-else Process

The program evaluates a condition.

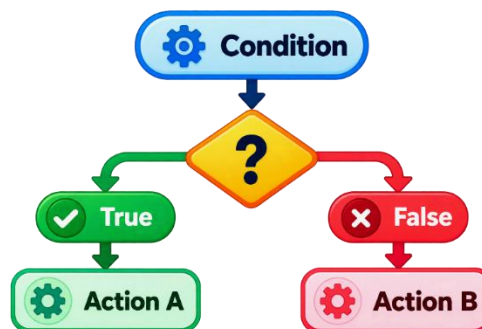


Figure 1: if-else

Only one of the two branches is executed.

14. The elif Statement

elif means “else if”.

It allows multiple conditions to be evaluated.

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Example:

```
score = 75

if score >= 80:
    print("Grade A")
elif score >= 70:
    print("Grade B")
else:
    print("Grade C")
```

Output:

Grade B

15. Multiple elif Statements

A program can contain several elif conditions.

Example:

```
score = 65

if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
elif score >= 60:
    print("Grade D")
else:
    print("Grade F")
```

Output:

Grade D

Python evaluates the conditions from top to bottom and executes the first matching branch.

16. Order of Conditions

The order of conditions is important.

Consider:

```
score = 85

if score >= 50:
    print("Pass")
elif score >= 80:
    print("Excellent")
```

The output is:

Pass

The first condition is already true, so Python does not evaluate the next branch.

A better structure is:

```
score = 85

if score >= 80:
    print("Excellent")
elif score >= 50:
    print("Pass")
else:
    print("Fail")
```

Output:

Excellent

17. Shorthand if

Python allows a simple if statement to be written on one line.

Normal:

```
a = 5
b = 10

if b > a:
    print("b is greater than a")
```

Shorthand:

```
if b > a: print("b is greater than a")
```

This form should be used only when the statement remains clear and readable.

18. Shorthand if-else

A simple conditional expression can also be written on one line.

Example:

```
a = 10
b = 5
```

```
print("A") if a > b else print("B")
```

Output:

A

A more useful form assigns a result:

```
age = 20

status = "Adult" if age >= 18 else "Minor"

print(status)
```

Output:

Adult

This syntax is commonly called a conditional expression.

19. Nested if Statements

A nested if is an if statement inside another conditional statement.

Example:

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Entry denied")
```

A nested structure is useful when the second decision should only be evaluated after the first condition is satisfied.

20. Multiple Conditions with and

Example:

```
age = 20
score = 80

if age >= 18 and score >= 50:
    print("Qualified")
```

Output:

Qualified

21. Multiple Conditions with or

Example:

```

score = 45
attendance = 90

if score >= 50 or attendance >= 80:
    print("Condition satisfied")

```

Output:

```

Condition satisfied

```

22. Combining Logical Operators

More than one logical operator can be used.

Example:

```

age = 20
score = 80
is_active = True

if age >= 18 and score >= 50 and is_active:
    print("Eligible")

```

For readability, parentheses may be used in more complex expressions:

```

if (age >= 18 and score >= 50) or is_active:
    print("Condition satisfied")

```

23. The pass Statement

Python does not allow an empty code block.

The pass statement acts as a placeholder.

Example:

```

score = 80

if score >= 50:
    pass

```

The program does nothing inside the if block, but the syntax remains valid.

Another example:

```

age = 20

if age >= 18:
    pass
else:
    print("Minor")

```

pass is useful when a code block will be implemented later.

24. Python Keywords Used in Conditional Statements

Important keywords include:

If elif else and or not pass True False

These keywords have special meanings and should not be used as ordinary variable names.

25. Initializing Variables

Initialization means creating a variable and assigning an initial value.

Example:

```
a = 5
b = 10
```

Here:

```
a → 5
b → 10
```

The values can then be used in conditional expressions.

26. Required Variable Comparison

Initialize:

```
a = 5
b = 10
```

Check whether b is greater than a:

```
if b > a:
    print("b is greater than a")
```

Output:

```
b is greater than a
```

The condition:

```
b > a
```

returns:

```
True
```

Therefore, the statement is displayed.

27. What Happens When the Condition is False?

Consider:

```
a = 20
b = 10

if b > a:
    print("b is greater than a")
```

Because:

```
10 > 20
```

is false, nothing is printed.

An else branch can handle this case:

```
a = 20
b = 10

if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

Output:

```
b is not greater than a
```

28. Comparing Two Numbers

A complete comparison program can determine whether the first number is greater, the second number is greater, or both are equal.

Example:

```
a = 10
b = 20

if a > b:
    print("a is greater than b")
elif b > a:
    print("b is greater than a")
else:
    print("a and b are equal")
```

Output:

```
b is greater than a
```

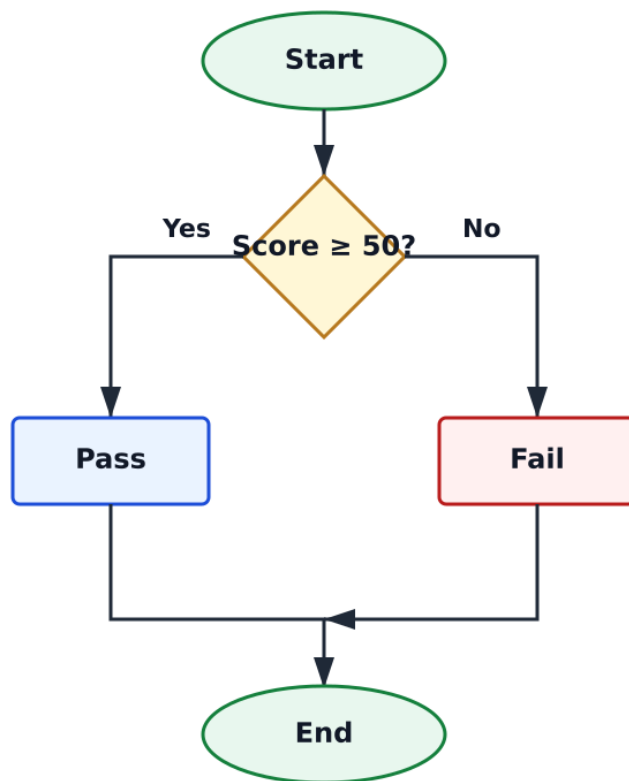
29. Practical Example — Pass or Fail

```
score = 70

if score >= 50:
    print("Pass")
else:
    print("Fail")
```

This decision has two possible outcomes:

Score Decision Flowchart



30. Practical Example — Positive, Negative or Zero

```
number = -5
```

```
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Output:

Negative

31. Practical Example — Student Grade

```
score = 85
```

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
```

```
print("Grade:", grade)
```

Output:

Grade: B

32. Practical Example — Login Decision

```
username = "admin"
password = "1234"

if username == "admin" and password == "1234":
    print("Login successful")
else:
    print("Invalid username or password")
```

This demonstrates multiple conditions with and.

33. Practical Example — Age Category

```
age = 25

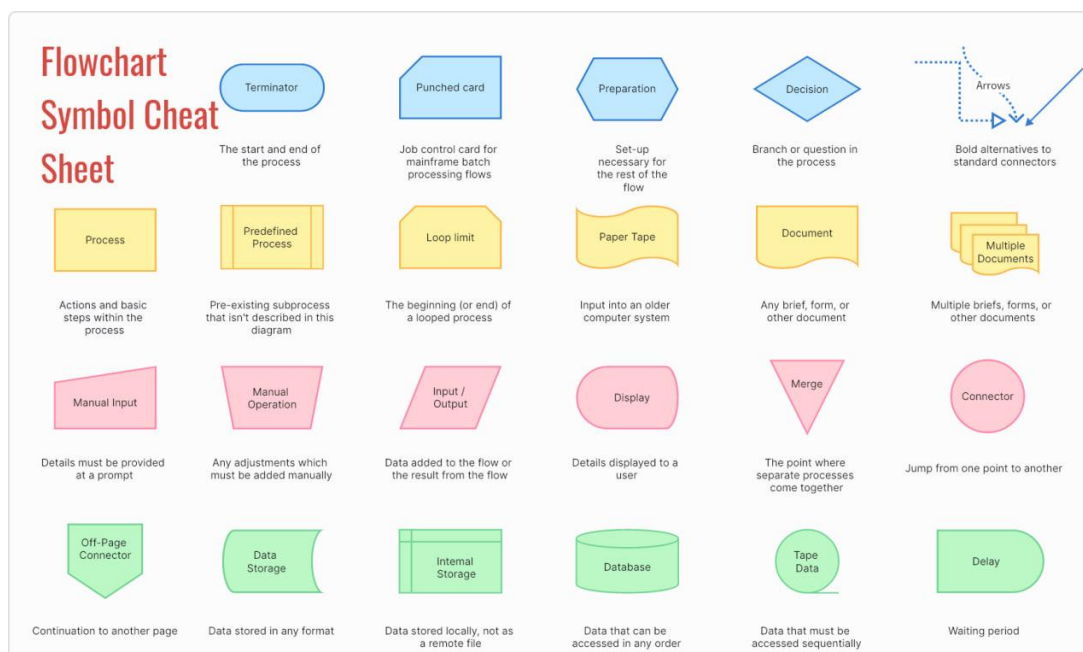
if age < 13:
    print("Child")
elif age < 18:
    print("Teenager")
elif age < 60:
    print("Adult")
else:
    print("Senior")
```

Output:

Adult

34. Flowcharts for Control Statements

A flowchart visually represents the steps and decisions in a program.



Common symbols include:

Symbol	Purpose
Oval	Start / End
Rectangle	Process
Diamond	Decision
Parallelogram	Input / Output
Arrow	Direction of flow

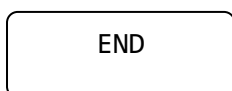
Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

35. Start/End Symbol

The oval represents:



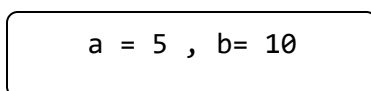
or:



36. Process Symbol

A rectangle represents processing.

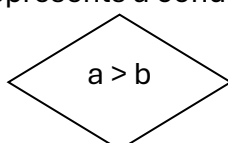
Example:



37. Decision Symbol

A diamond represents a condition or decision.

Example:



The decision normally has two branches:

True / Yes

False / No

38. Input/Output Symbol

A parallelogram represents input or output.

For example:

Print "b is greater than a"

is an output operation.

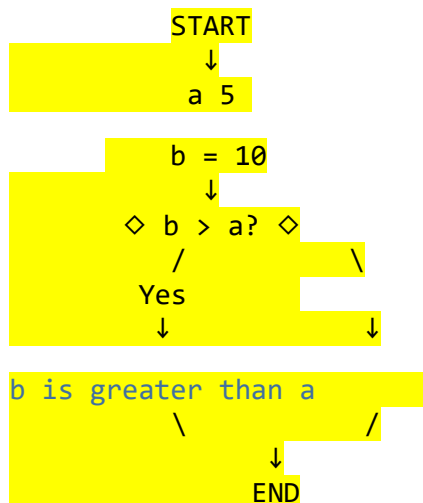
39. Flowchart for Required LO5 Program

Program:

```
a = 5
b = 10

if b > a:
    print("b is greater than a")
```

Process flow:



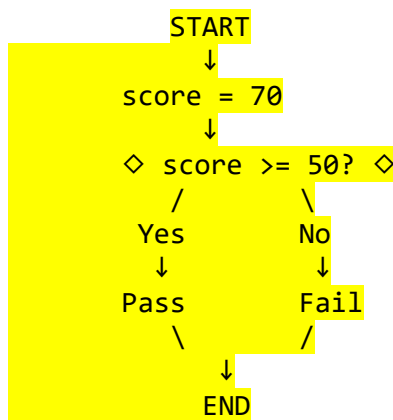
40. Flowchart for if-else

Example:

```
score = 70

if score >= 50:
    print("Pass")
else:
    print("Fail")
```

Flow:

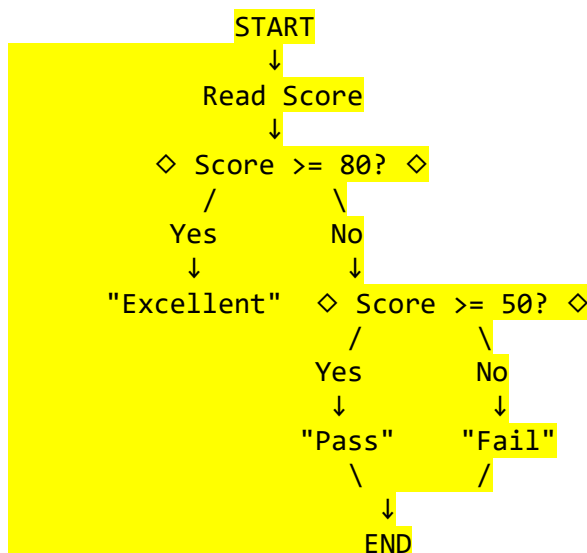


41. Flowchart for if-elif-else

Example logic:

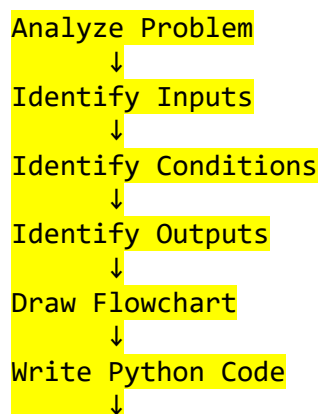
Score >= 80 → Excellent
Score >= 50 → Pass
Otherwise → Fail

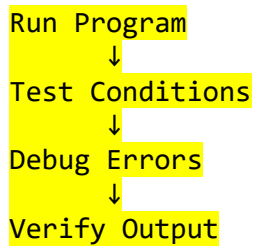
Flow:



42. Developing a Control-Flow Program

A simple development process is:





43. Testing Conditional Programs

Conditional programs should be tested using different cases.

For example:

```
if b > a:
    print("b is greater than a")
elif a > b:
    print("a is greater than b")
else:
    print("a and b are equal")
```

Test at least:

Test	a	b	Expected Result
1	5	10	b is greater than a
2	10	5	a is greater than b
3	10	10	a and b are equal

Testing all major branches helps verify program correctness.

44. Common Conditional Statement Errors

Missing Colon

Incorrect:

```
if age >= 18
    print("Adult")
```

Correct:

```
if age >= 18:
    print("Adult")
```

Incorrect Indentation

Incorrect:

```
if age >= 18:
print("Adult")
```

Correct:

```
if age >= 18:
    print("Adult")
```

Assignment Instead of Comparison

Incorrect:

```
if age = 18:
```

Correct:

```
if age == 18:  
    print("Age is 18")
```

Remember:

```
=    assignment  
==   comparison
```

Incorrect Keyword

Incorrect:

```
elseif
```

Python uses:

```
elif
```

45. Debugging Control-Flow Programs

When a conditional program produces an error or incorrect result:

1. Check variable values.
2. Check comparison operators.
3. Check logical operators.
4. Check the colon.
5. Check indentation.
6. Check condition order.
7. Check if, elif, and else structure.
8. Run different test cases.
9. Compare actual output with expected output.
10. Correct the program and run it again.

SELF-CHECK No. 1

Part A — Multiple Choice

1. What is control flow?

- A. Computer hardware
- B. The order in which program statements execute
- C. A Python variable
- D. A Python string

2. Which keyword begins a conditional statement?

- A. for
- B. if
- C. print
- D. input

3. Which operator means greater than?

- A. <
- B. >
- C. ==
- D. !=

4. Which operator checks equality?

- A. =
- B. ==
- C. >=
- D. <=

5. Which is a logical operator?

- A. and
- B. print
- C. input
- D. type

6. Which logical operator requires both conditions to be true?

- A. or
- B. not
- C. and
- D. if

7. Which logical operator requires at least one condition to be true?

- A. or
- B. and
- C. not
- D. else

8. Which keyword is used for an additional condition?

- A. elseif
- B. else if
- C. elif
- D. then

9. Which keyword handles the remaining alternative?

- A. if
- B. elif
- C. else
- D. pass

10. Which statement acts as a placeholder?

- A. print
- B. pass
- C. input
- D. break

11. Which flowchart symbol represents a decision?

- A. Rectangle
- B. Diamond
- C. Oval
- D. Arrow

12. Which symbol represents Start/End?

- A. Oval
- B. Diamond
- C. Rectangle
- D. Parallelogram

13. Given a = 5 and b = 10, what is b > a?

- A. False
- B. True
- C. 5
- D. 10

14. Which is valid Python?

- A. if b > a:
- B. if b > a then
- C. if (b > a) then
- D. IF b > a

15. What is the correct Python keyword for “else if”?

- A. elseif
- B. elif
- C. else_if
- D. ifelse

ANSWER KEY No. 5.1

Part A

No.	Answer
1	B
2	B
3	B
4	B
5	A
6	C
7	A
8	C
9	C
10	B
11	B
12	A
13	B
14	A
15	B

OPERATION SHEET No. 5.1

Title: Create and Execute Python Conditional Statements

Performance Objective

Given a computer with Python installed and an approved Python development environment, the trainee must initialize variables, apply comparison and logical operators, create if, elif, and else statements, execute the program, test different conditions and verify the results.

Description

This operation demonstrates how to create and execute a basic Python decision-making program.

Picture/Overview

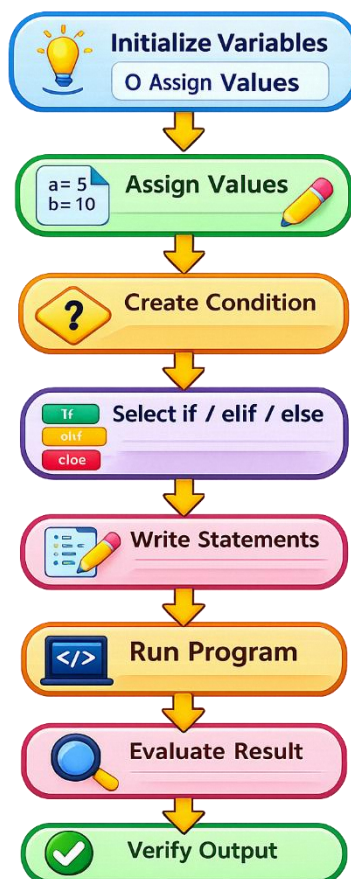


Figure 2:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/IDE	Software	1
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference	Copy	1
7	CBLM/Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
-----	----------------	-----------------

No.	Procedure/Step	Key Performance
1	Open Python IDLE/IDE	Development environment opens correctly
2	Create new Python file	Source file is created
3	Initialize variables a and b	Variables are correctly created
4	Assign values	Appropriate values are stored
5	Create comparison	Correct comparison operator is used
6	Create if statement	Correct syntax and colon are used
7	Apply indentation	Code block is correctly indented
8	Display true result	Required message is printed
9	Add elif and else	Alternative conditions are correctly structured
10	Apply logical operators	and, or, or not is correctly used
11	Save program	.py extension is used
12	Execute program	Program executes successfully
13	Test different values	Major branches are tested
14	Verify output	Results match conditions

Operation Procedure

Create:

```
a = 5
b = 10
```

Check:

```
if b > a:
    print("b is greater than a")
```

Extend:

```
if a > b:
    print("a is greater than b")
elif b > a:
    print("b is greater than a")
else:
    print("a and b are equal")
```

Test:

```
a = 5, b = 10
a = 10, b = 5
a = 10, b = 10
```

Save as:

```
control_flow.py
```

Run and verify all three test cases.

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 5.1

Title: Apply Logical Conditions and Operators

Performance Task

Create logical conditions using comparison and logical operators.

Performance Objective

Given variable values, the trainee must correctly apply comparison operators and the logical operators and, or, and not.

Description

Use:

a = 5
b = 10

to evaluate different conditions.

Picture/Overview

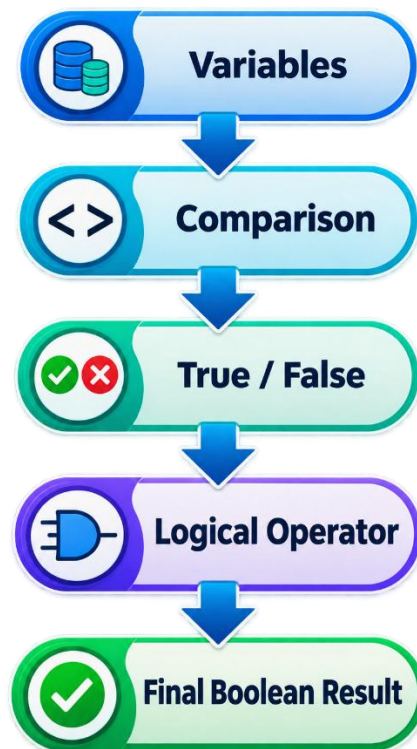


Figure 3:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
	1. Initialize variables	Correct variables are created
	2. Apply > and <	Results are correct
	3. Apply == and !=	Results are correct
	4. Apply >= and <=	Results are correct

No.	Procedure/Step	Key Performance
	5. Apply and	Correct Boolean result is produced
	6. Apply or	Correct Boolean result is produced
	7. Apply not	Boolean result is reversed correctly
	8. Verify results	Actual results match expected results

Practical Exercise

a = 5

b = 10

```
print(a > b)
```

```
print(a < b)
```

```
print(a == b)
```

```
print(a != b)
```

```
print(a <= b)
```

```
print(a >= b)
```

```
print(a < b and b == 10)
```

```
print(a > b or b == 10)
```

```
print(not a > b)
```

Complete:

Expression	Expected Result	Actual Result
a > b	False	_____
a < b	True	_____
a == b	False	_____
a != b	True	_____
a <= b	True	_____
a >= b	False	_____
a < b and b == 10	True	_____
a > b or b == 10	True	_____
not a > b	True	_____

Assessment Method

- Direct observation
- Practical exercise
- Worksheet review
- Output verification

TASK SHEET No. 5.2

Title: Develop if and if-else Statements

Performance Task

Develop a Python program using if and if-else.

Performance Objective

Given two initialized variables, the trainee must check whether b is greater than a and display the appropriate result.

Description

The trainee will practice the core LO5 requirement:

a = 5

b = 10

and evaluate:

b > a

Picture/Overview

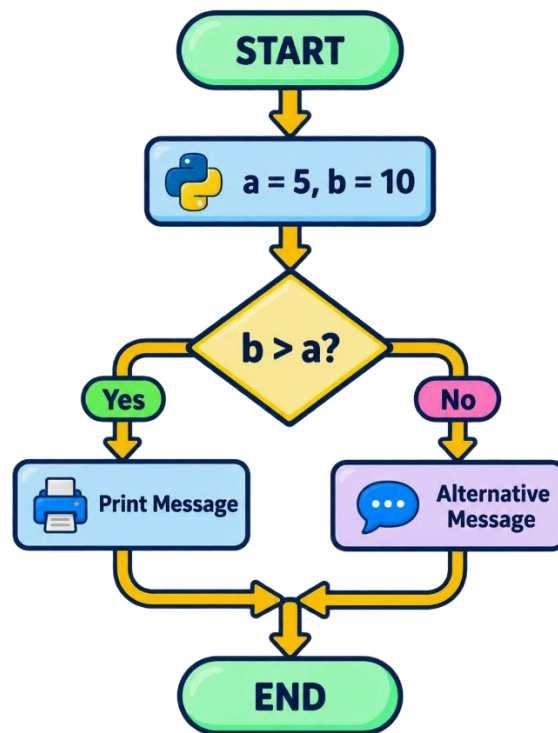


Figure 4:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create a	Variable is initialized
2	Create b	Variable is initialized
3	Assign values	a=5, b=10
4	Create if	Correct syntax is applied

No.	Procedure/Step	Key Performance
5	Evaluate $b > a$	Correct condition is used
6	Print true message	Required message appears
7	Add else	False branch is handled
8	Test different values	Both outcomes are tested

Practical Exercise A

```
a = 5
b = 10
```

```
if b > a:
    print("b is greater than a")
```

Practical Exercise B

Modify:

```
a = 20
b = 10
```

Use:

```
if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

Record results:

a	b	Expected Result	Actual Result
5	10	b is greater than a	_____
20	10	b is not greater than a	_____

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 5.3

Title: Develop if-elif-else Decision Statements

Performance Task

Develop a Python program that evaluates multiple conditions.

Performance Objective

Given a numeric score, the trainee must use `if`, `elif`, and `else` to classify the result according to specified conditions.

Description

Create a simple grading decision.

Picture/Overview

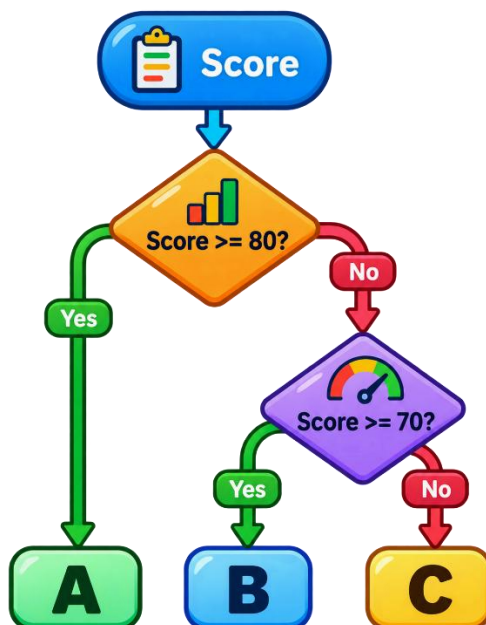


Figure 5:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create score variable	Correct value is assigned
2	Create first <code>if</code>	Highest condition is evaluated
3	Add <code>elif</code>	Additional condition is evaluated
4	Add <code>else</code>	Remaining case is handled
5	Apply indentation	Blocks are correctly structured
6	Execute program	Program runs successfully
7	Test multiple scores	Different branches are verified

Practical Exercise

Develop:

```
score = 75
```

```
if score >= 80:  
    print("Grade A")  
elif score >= 70:  
    print("Grade B")  
else:  
    print("Grade C")
```

Test:

Score	Expected Result	Actual Result
85	Grade A	_____
75	Grade B	_____
60	Grade C	_____

Assessment Method

- Direct observation
- Practical exercise
- Program execution
- Output verification

TASK SHEET No. 5.4

Title: Apply Shorthand and Nested Conditional Statements

Performance Task

Develop shorthand and nested Python conditional statements.

Performance Objective

Given simple decision requirements, the trainee must apply shorthand conditional expressions and nested conditional statements where appropriate.

Description

This activity introduces compact and nested decision structures.

Picture/Overview

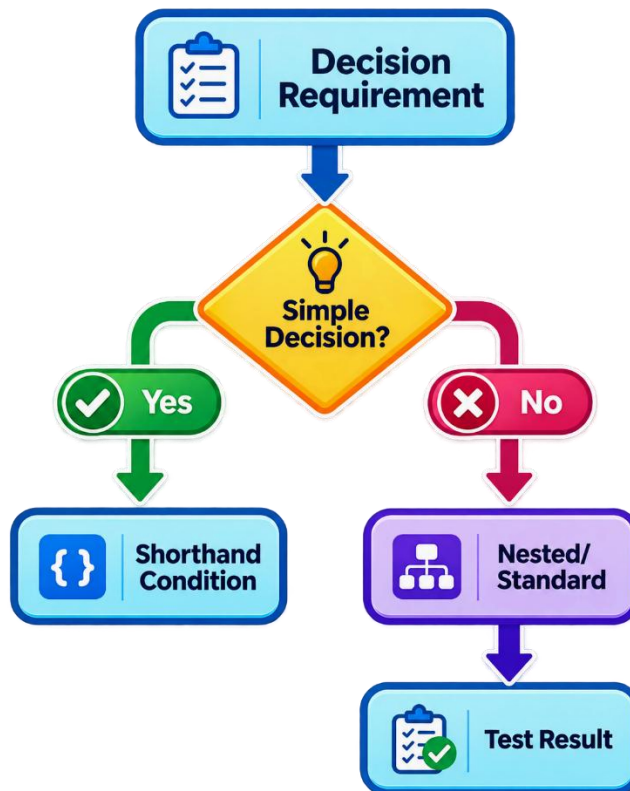


Figure 6:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Identify decision requirement	Appropriate structure is selected
2	Apply shorthand <code>if</code>	One-line condition is correct
3	Apply conditional expression	True/false values are correct
4	Create outer condition	First decision is correct
5	Create nested condition	Second decision is correctly nested
6	Apply indentation	Nested blocks are correct

No.	Procedure/Step	Key Performance
7	Execute and test	Required branches are verified

Practical Exercise A — Shorthand if

```
a = 5
b = 10
```

```
if b > a: print("b is greater than a")
```

Practical Exercise B — Conditional Expression

```
age = 20
```

```
status = "Adult" if age >= 18 else "Minor"
```

```
print(status)
```

Practical Exercise C — Nested if

```
age = 20
has_id = True
```

```
if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Entry denied")
```

Test:

Age	Has ID	Expected Result
20	True	Entry allowed
20	False	ID required
16	True	Entry denied

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Output verification

TASK SHEET No. 5.5

Title: Draw Process Flow Diagrams for Control Statements

Performance Task

Draw process flow diagrams representing if, if-else, and if-elif-else decision structures.

Performance Objective

Given conditional program requirements, the trainee must identify appropriate flowchart symbols, arrange them logically, connect them using arrows and correctly represent true and false decision branches.

Description

The trainee converts Python decision logic into a graphical process representation.

Picture/Overview

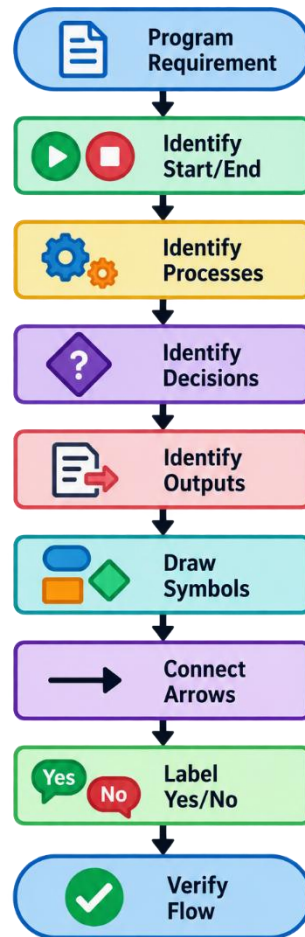


Figure 7:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Flowchart Worksheet	Copy	1
2	Pencil/Pen	Unit	1
3	Ruler	Unit	1
4	Computer/Laptop	Unit	1
5	Flowchart Software, if available	Copy	1
6	CBLM	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze program	Logic is correctly identified
2	Draw Start symbol	Oval is correctly used
3	Draw process	Rectangle is correctly used
4	Draw decision	Diamond is correctly used
5	Draw output	Appropriate symbol is used
6	Connect symbols	Arrows show correct direction
7	Label decision branches	Yes/No or True/False is shown
8	Draw End symbol	Flow terminates correctly
9	Compare with Python logic	Diagram matches program

Exercise A

Draw a flowchart for:

```
a = 5
b = 10

if b > a:
    print("b is greater than a")
```

Exercise B

Draw a flowchart for:

```
score = 70

if score >= 50:
    print("Pass")
else:
    print("Fail")
```

Exercise C

Draw a flowchart for:

```
number = 0

if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Assessment Method

- Flowchart review
- Direct observation
- Oral questioning
- Practical assessment

JOB SHEET No. 5.1

Title: Develop a Student Result Decision Program

Performance Task

Develop a complete Python program that applies control-flow statements to evaluate a student's result and eligibility status.

Performance Objective

Given a computer, Python development environment and program requirements, the trainee must analyze the decision problem, draw a process flow diagram, initialize variables, apply comparison and logical operators, implement if, elif, else and nested conditions, execute and test the program, debug errors and verify the final output.

Description

Develop:

student_result_decision.py

The program will evaluate student information and demonstrate the major control-flow concepts covered in LO5.

Picture/Overview

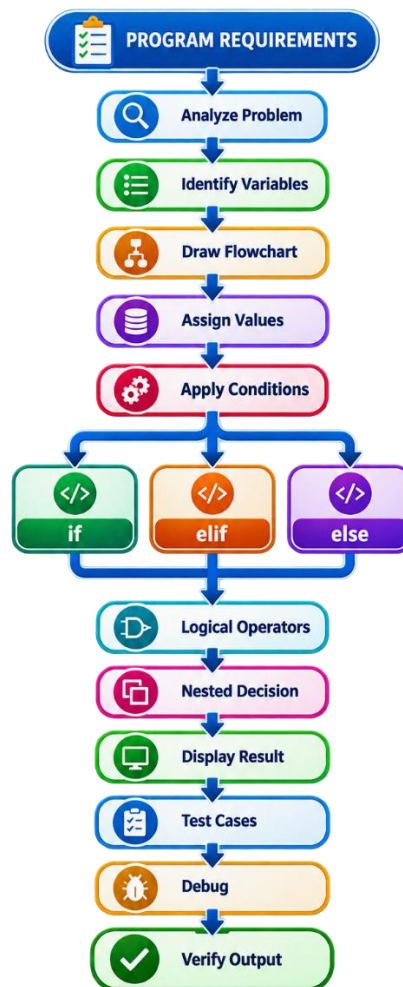


Figure 8:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/IDE	Software	1

No.	Description	Unit	Qty
5	Programming Worksheet	Copy	1
6	Flowchart Worksheet	Copy	1
7	Python Reference Material	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze requirements	Inputs, conditions and outputs are identified
2	Draw flowchart	Decision logic is represented correctly
3	Open Python IDE	Development environment opens
4	Create program file	Source file is created
5	Initialize variables	Required variables are created
6	Assign values	Appropriate values are stored
7	Apply comparison operators	Conditions are correctly constructed
8	Apply logical operators	Multiple conditions are correctly evaluated
9	Create if	First decision is correctly implemented
10	Create elif	Additional decisions are correctly implemented
11	Create else	Remaining case is handled
12	Apply nested condition	Nested decision operates correctly
13	Apply shorthand condition	Simple conditional expression is correctly demonstrated
14	Save program	.py extension is used
15	Execute program	Program runs
16	Test different values	All major branches are tested
17	Identify errors	Syntax/logical errors are recognized
18	Debug program	Errors are corrected
19	Re-execute program	Corrected program runs
20	Verify output	Results match requirements

Program Requirements

The program must contain:

1. Student name.
2. Student score.
3. Student attendance.
4. if.
5. elif.
6. else.
7. Comparison operators.
8. Logical operators.
9. Nested conditional statement.
10. Shorthand conditional expression.
11. Appropriate output statements.
12. At least three test cases.

Part 1 — Variables

Create:

```
student_name = "Dara"
score = 85
attendance = 90
```

Part 2 — Determine Grade

Use:

90–100 → Grade A
80–89 → Grade B
70–79 → Grade C
60–69 → Grade D
Below 60 → Grade F

Part 3 — Determine Pass/Fail

A student passes when:

Score >= 60
AND
Attendance >= 75

Part 4 — Nested Condition

If the student passes, check whether:

Score >= 90

If true:

Excellent Performance

Otherwise:

Satisfactory Performance

Part 5 — Shorthand Condition

Use a conditional expression to determine:

Eligible

or:

Not Eligible

based on attendance.

Example Program

```
# Student Result Decision Program
```

```
student_name = "Dara"  
score = 85  
attendance = 90  
  
print("STUDENT RESULT")  
print("Name:", student_name)  
print("Score:", score)  
print("Attendance:", attendance)
```

```
# Determine grade
```

```
if score >= 90:  
    grade = "A"  
elif score >= 80:  
    grade = "B"  
elif score >= 70:  
    grade = "C"  
elif score >= 60:  
    grade = "D"  
else:  
    grade = "F"
```

```
print("Grade:", grade)
```

```
# Determine pass/fail
```

```
if score >= 60 and attendance >= 75:
```

```

print("Result: Pass")

# Nested condition
if score >= 90:
    print("Performance: Excellent")
else:
    print("Performance: Satisfactory")
else:
    print("Result: Fail")

# Shorthand conditional expression
eligibility = "Eligible" if attendance >= 75 else "Not Eligible"

print("Eligibility:", eligibility)

```

Expected Output for Test Case 1

For:

```

student_name = Dara
score = 85
attendance = 90

```

Expected:

```

STUDENT RESULT
Name: Dara
Score: 85
Attendance: 90
Grade: B
Result: Pass
Performance: Satisfactory
Eligibility: Eligible

```

Test Cases

Test	Score	Attendance	Expected Grade	Expected Result	Eligibility
1	95	90	A	Pass	Eligible
2	75	80	C	Pass	Eligible
3	55	90	F	Fail	Eligible
4	85	60	B	Fail	Not Eligible

The trainee should execute the program using each test case and record the actual result.

WORKSHEET No. 5.2

Title: Python Control Flow Practice and Verification Worksheet

Trainee Name: _____

Date: _____

Part A — Comparison Operators

Complete the table.

Operator	Meaning
==	_____
!=	_____
>	_____
<	_____
>=	_____
<=	_____

Part B — Logical Operators

Complete:

Operator	Meaning
and	_____
or	_____
not	_____

Part C — Evaluate Conditions

Given:

```
a = 5
b = 10
```

Determine:

Condition	True/False
b > a	_____
a > b	_____
a == b	_____
a != b	_____
a < b and b == 10	_____
a > b or b == 10	_____
not a > b	_____

Part D — Complete the if Statement

Complete:

```
a = 5
b = 10
```

```
_____ b > a:
    print("b is greater than a")
```

Answer: _____

Part E — Complete if-else

```
score = 45
```

```
if score >= 50:
    print("Pass")
    _____:
        print("Fail")
```

Answer: _____

Part F — Complete if-elif-else

```
score = 75
```

```
if score >= 80:
    print("Grade A")
    _____ score >= 70:
        print("Grade B")
    _____:
        print("Grade C")
```

Answers:

Part G — Shorthand Conditional Expression

Complete:

```
age = 20
```

```
status = "Adult" _____ age >= 18 _____ "Minor"
```

Answers:

Part H — Flowchart Symbols

Complete:

Purpose	Symbol
Start/End	_____
Process	_____
Decision	_____
Input/Output	_____
Direction of flow	_____

Part I — Debugging

Correct:

```
age = 20
```

```
if age >= 18
    print("Adult")
```

Correct code:

Correct:

```
score = 80
```

```
if score >= 50:
    print("Pass")
```

Correct code:

PERFORMANCE CHECKLIST No. 5.1

Title: Develop a Program Based on Control Flow Statements

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Identified Python decision-making concepts	☐	☐
2	Identified logical conditions	☐	☐
3	Identified comparison operators	☐	☐
4	Applied == correctly	☐	☐
5	Applied != correctly	☐	☐
6	Applied > and < correctly	☐	☐
7	Applied >= and <= correctly	☐	☐
8	Identified logical operators	☐	☐
9	Applied and correctly	☐	☐
10	Applied or correctly	☐	☐
11	Applied not correctly	☐	☐
12	Applied an if statement correctly	☐	☐
13	Applied an if-else structure correctly	☐	☐
14	Applied an if-elif-else structure correctly	☐	☐
15	Applied multiple elif conditions appropriately	☐	☐
16	Applied shorthand if appropriately	☐	☐
17	Applied shorthand conditional expression appropriately	☐	☐
18	Initialized variable a	☐	☐
19	Initialized variable b	☐	☐
20	Assigned values to variables correctly	☐	☐
21	Checked whether b is greater than a	☐	☐
22	Printed the required statement when the condition was true	☐	☐
23	Applied correct indentation	☐	☐
24	Applied nested conditional statements	☐	☐
25	Applied pass where appropriate	☐	☐
26	Selected an appropriate conditional structure for the problem	☐	☐
27	Identified flowchart Start/End symbol	☐	☐
28	Identified flowchart Process symbol	☐	☐
29	Identified flowchart Decision symbol	☐	☐
30	Identified flowchart Input/Output symbol	☐	☐
31	Drew arrows showing correct process direction	☐	☐

No.	Criteria	Yes	No
32	Labeled True/False or Yes/No decision branches	☐	☐
33	Drew a process flow diagram for an if statement	☐	☐
34	Drew a process flow diagram for if-else	☐	☐
35	Drew a process flow diagram for multiple conditions	☐	☐
36	Flowchart matched the Python program logic	☐	☐
37	Saved the program with .py extension	☐	☐
38	Executed the Python program successfully	☐	☐
39	Tested true and false conditions	☐	☐
40	Tested multiple decision branches	☐	☐
41	Identified basic syntax errors	☐	☐
42	Identified basic logical errors	☐	☐
43	Corrected identified errors	☐	☐
44	Verified final program output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____